

DevOps Fundamentals

Compact Course — Day 1

Culture, Git Workflows & CI Pipelines

Day 1 — Agenda

Part 1 — What is DevOps?

- Silos vs. cross-functional teams
- The Three Ways of DevOps
- DORA metrics: measuring delivery performance
- The DevOps toolchain
- Shift-left: quality and security earlier

Part 2 — Git Branching Strategies

- Git refresh: commit, branch, remote, PR/MR
- GitFlow, GitHub Flow, GitLab Flow, and Trunk-Based Development
- Branch protection rules and PR templates
- Semantic versioning and conventional commits
- Monorepo vs. polyrepo

Part 3 — CI/CD Fundamentals

- What is CI? What is CD (Delivery vs. Deployment)?
- The CI/CD spectrum
- Pipeline anatomy: trigger → runner → jobs → steps → artifacts
- GitHub-hosted and self-hosted runners

Part 4 — GitHub Actions

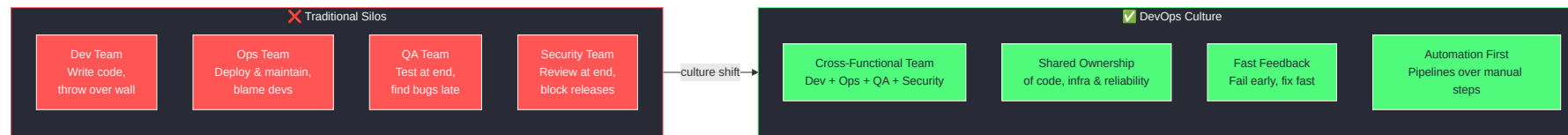
- Workflow file structure (`.github/workflows/*.yml`)
- Triggers, jobs, steps, and the Actions marketplace
- Secrets, environment variables, caching, and matrix builds
- Artifacts and job dependencies
- Reusable workflows

Part 5 — GitLab CI

- `.gitlab-ci.yml` anatomy
- Stages, jobs, runners, variables, artifacts, and caching
- `include:` and `extends:` for DRY pipelines
- Manual gates and environment deployments
- GitHub Actions vs. GitLab CI comparison

Part 1 — What is DevOps?

The Problem: Silos



Silos Create Pain

Dev Team:

- "It works on my machine" — ships code, moves on
- Prioritises features over stability

Ops Team:

- Responsible for stability, not delivery speed
- Infrastructure changes are rare and risky

Result:

- Long release cycles (weeks or months)
- Blame culture when things break
- Manual, error-prone deployments

No shared visibility

DevOps: A Culture Shift

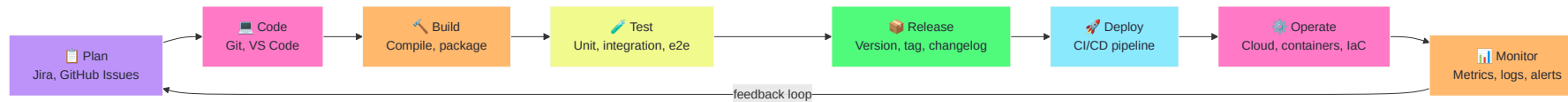
DevOps is not a tool, a job title, or a team.

It is a **culture** of shared ownership, fast feedback, and continuous improvement.

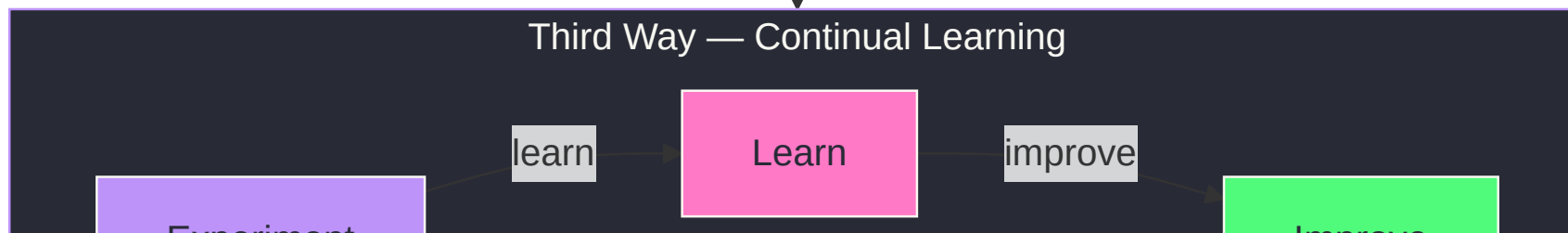
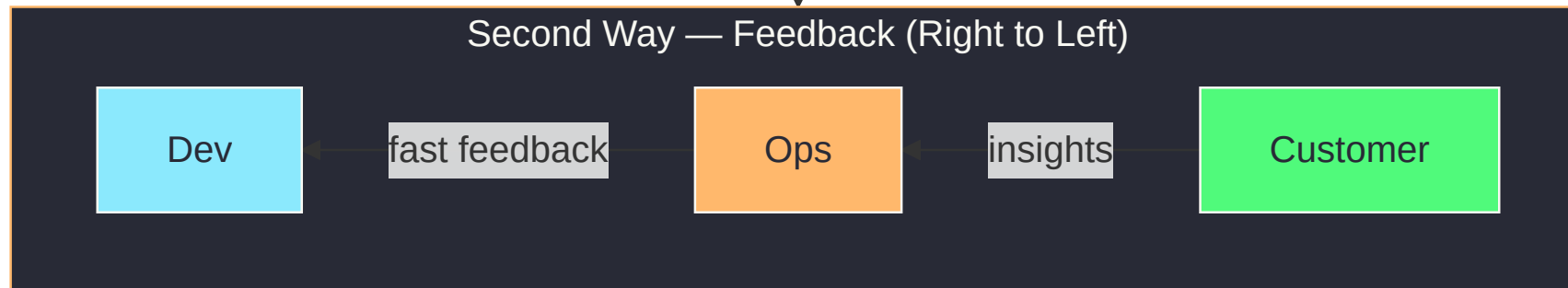
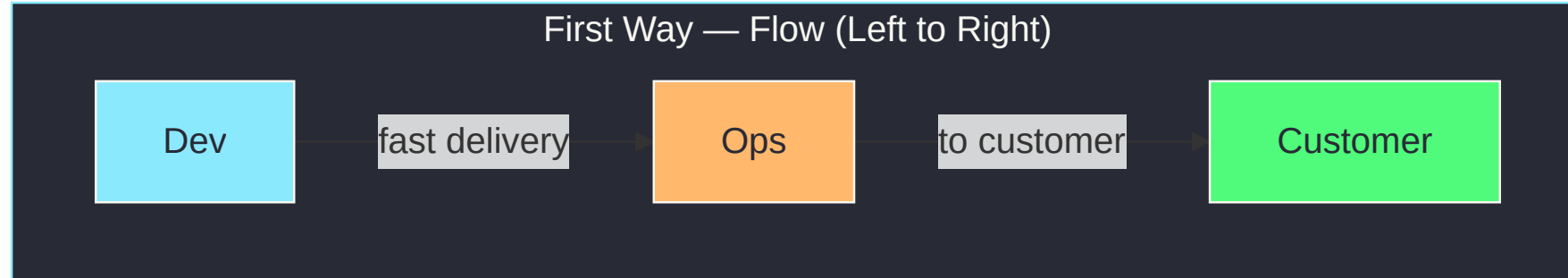
Core principles:

- Dev, Ops, QA, and Security work **together** across the full lifecycle
- You build it, you run it
- Small, frequent changes over large, risky releases
- Automate everything that can be automated

The DevOps Infinity Loop



The Three Ways



The Three Ways — Explained

Way	Description	Example
Flow	Maximise the speed of delivery from left to right	Deploy multiple times a day
Feedback	Create fast feedback loops from right to left	Tests run in under 5 minutes
Continual Learning	Foster experimentation and improvement	Blameless postmortems

These principles underpin everything in DevOps practice.

DORA Metrics — Are You Improving?

Low Performers (avoid)

Deployment Frequency
Less than once per 6
months

Lead Time for Changes
More than 6 months

Mean Time to Restore
More than 1 week

Elite Performers (target)

Deployment Frequency
Multiple times per day

Lead Time for Changes
Less than 1 hour

Mean Time to Restore
Less than 1 hour

DORA Metrics — Explained

Metric	What it measures	Elite target
Deployment Frequency	How often you deploy to production	Multiple times per day
Lead Time for Changes	Commit → running in production	Less than 1 hour
Mean Time to Restore (MTTR)	Time to recover from an incident	Less than 1 hour
Change Failure Rate	% of deployments causing incidents	0–15%

Measure these to know whether your DevOps practices are actually working.

The DevOps Toolchain

Each phase of the loop has tools:

Phase	Tools
Plan	GitHub Issues, Jira, Linear
Code	VS Code, Git
Build	npm, Maven, Docker
Test	Jest, pytest, Playwright
Release	GitHub Releases, semantic-release
Deploy	GitHub Actions, GitLab CI, ArgoCD
Operate	Kubernetes, Terraform, Ansible
Monitor	Prometheus, Grafana, Datadog

Shift-Left: Catch Problems Earlier

Traditional approach:

```
Code → Code → Code → Code → (finally) Test → (maybe) Security scan → Release
```

Shift-left approach:

```
Code →  Unit test  
→  Lint + type check  
→  Security scan (SAST)  
→  Dependency audit  
→ Merge → Integration tests → Deploy
```

The earlier you catch a bug, the cheaper it is to fix.

A bug caught in CI costs 10× less to fix than one found in production.

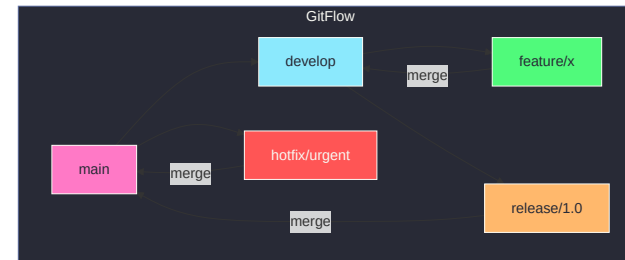
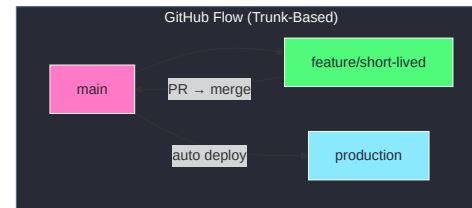
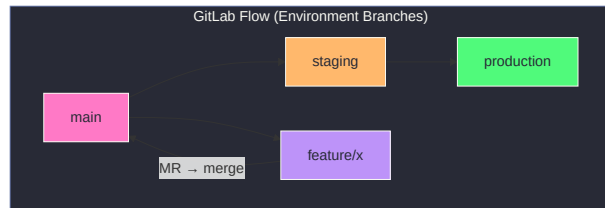
Part 2 — Git Branching Strategies

Git Refresh

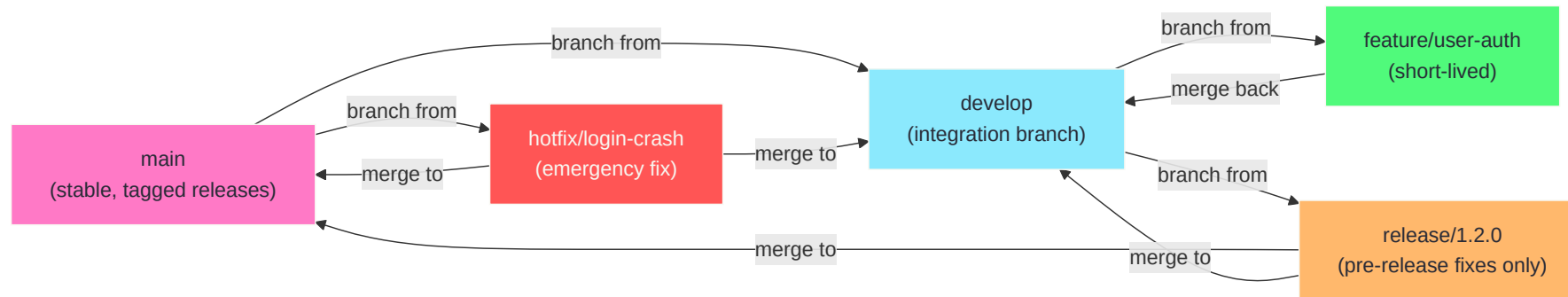
The concepts you need for branching strategies:

Concept	Description
<code>commit</code>	A snapshot of changes with a hash
<code>branch</code>	A pointer to a commit — cheap to create
<code>remote</code>	A copy of the repo on a server (GitHub / GitLab)
<code>PR / MR</code>	Pull Request / Merge Request — code review mechanism
<code>merge</code>	Combine two branches
<code>rebase</code>	Move commits onto a different base commit
<code>tag</code>	An immutable pointer to a specific commit (usually a release)

Branching Strategies Overview



GitFlow



GitFlow — When to Use It

Good for:

- Scheduled releases (mobile apps, on-premises software)
- Multiple versions maintained simultaneously
- Large teams with clear release ownership

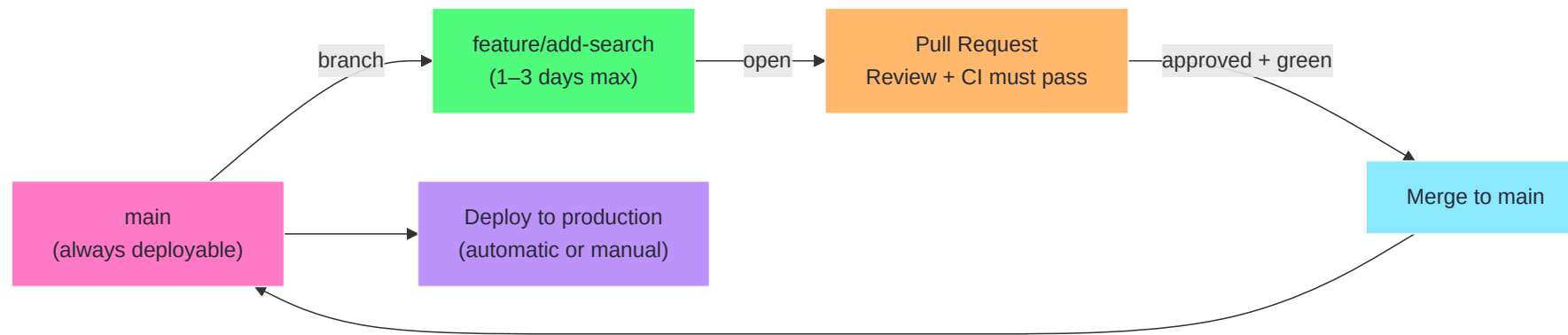
Drawbacks:

- Complex — many branches, many merge operations
- Long-lived branches → large merge conflicts
- Slows down delivery velocity

Branches:

Branch	Purpose

GitHub Flow (Trunk-Based)



GitHub Flow — When to Use It

Good for:

- Web applications with continuous deployment
- Small to medium teams
- When you want to deploy frequently

Rules:

1. `main` is always deployable
2. Create a branch for every change (name it descriptively)
3. Open a PR as soon as you push
4. Deploy to staging from the branch to validate
5. Merge to main only after review and green CI
6. Deploy to production immediately after merge

GitLab Flow — Environment Branches

Extends GitHub Flow with environment-tracking branches:

```
feature/x → main → staging → production
```

- Each environment branch tracks what is deployed where
- Merging to `production` = deploying to production
- Suits teams that do not auto-deploy immediately from main

Trunk-Based Development

The extreme of GitHub Flow:

- Everyone commits directly to `main` (or merges PRs within 24 hours)
- Feature flags hide incomplete work in production
- No long-lived branches at all

Requires:

- Strong automated test coverage
- Feature flag infrastructure
- High team discipline

Used by Google, Facebook, Netflix internally.

Branch Protection Rules

Always protect `main` (and `develop` in GitFlow):

GitHub:

Settings → Branches → Add branch protection rule

- ☒ Require pull request reviews (min. 1)
- ☒ Require status checks to pass (CI must be green)
- ☒ Require branches to be up to date before merging
- ☒ Restrict who can push to matching branches

GitLab:

Settings → Repository → Protected Branches

Allowed to merge: Developers + Maintainers

Allowed to push: No one (force via MR)

PR Templates

Create `.github/pull_request_template.md` :

```
## What does this PR do?
<!-- Brief description -->

## How to test
<!-- Steps to validate the change -->

## Checklist
- [ ] Tests added or updated
- [ ] Documentation updated if needed
- [ ] No secrets committed
- [ ] Dependent PRs linked
```

GitLab: create `.gitlab/merge_request_templates/Default.md`

Semantic Versioning

Format: MAJOR.MINOR.PATCH

Increment	When	Example
MAJOR	Breaking change — API incompatibility	1.0.0 → 2.0.0
MINOR	New feature — backwards compatible	1.0.0 → 1.1.0
PATCH	Bug fix — backwards compatible	1.0.0 → 1.0.1

Pre-releases: 1.1.0-beta.1 , 2.0.0-rc.3

Conventional Commits

Format: `<type>[optional scope]: <description>`

```
feat: add user authentication
fix: correct calculation in invoice total
chore: update dependencies
docs: update README installation steps
refactor: extract payment logic into service
test: add unit tests for user service
ci: add caching to GitHub Actions workflow

feat!: redesign authentication API    ← breaking change (MAJOR)
```

Benefits:

- Auto-generate changelogs
- Drive semantic-release automation
- Make `git log` readable

Monorepo vs. Polyrepo

	Monorepo	Polyrepo
Definition	All services in one repo	One repo per service
Examples	Google, Meta, Nx, Turborepo	Classic microservices
Pros	Atomic cross-service changes, shared tooling	Independent deployments, clear ownership
Cons	CI can be slow without path filtering, complex	Code sharing is harder, version management
CI tip	Use <code>paths:</code> filters in workflows to only run affected pipelines	

Part 3 — CI/CD Fundamentals

What is CI/CD?

Continuous Integration (CI):

- Every commit triggers an automated build and test run
- Goal: detect problems within minutes, not days
- Prevents "integration hell" — merging weeks of diverged work

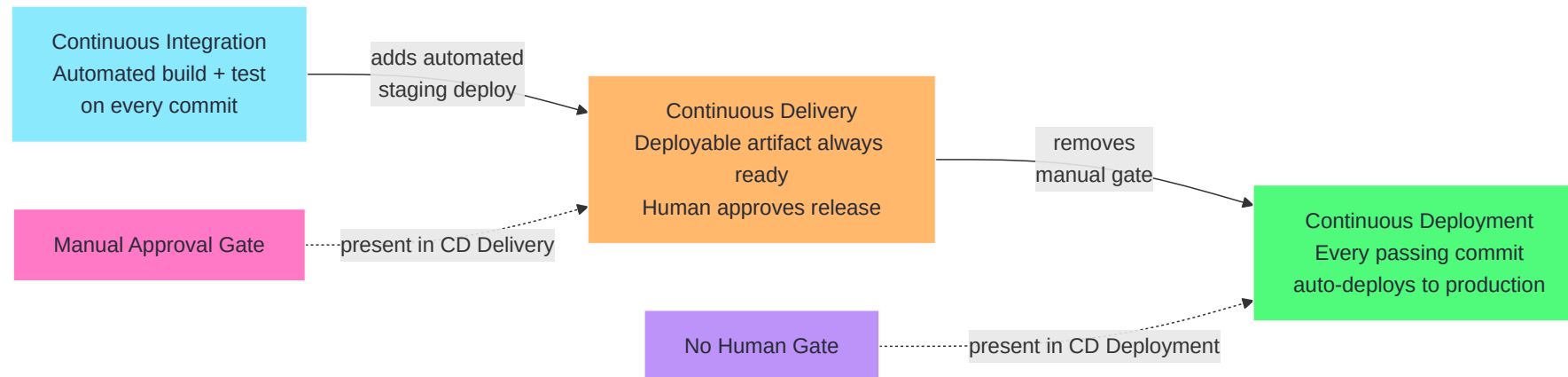
Continuous Delivery (CD — Delivery):

- Every commit that passes CI is deployed to a staging/pre-prod environment
- A human approves the final release to production

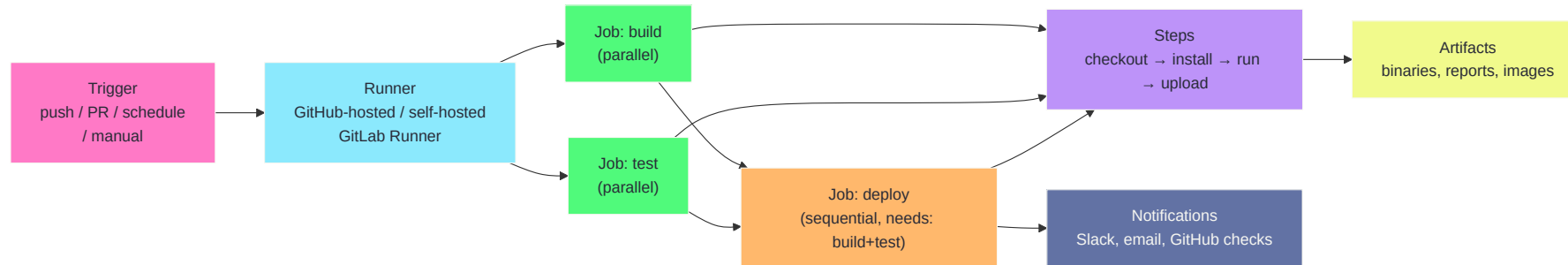
Continuous Deployment (CD — Deployment):

- Every commit that passes CI is deployed automatically all the way to production
- No human gates

The CI/CD Spectrum



Pipeline Anatomy



Pipeline Anatomy — Concepts

Concept	Description
Trigger	What starts the pipeline (push, PR, schedule, manual)
Runner	The machine/container that executes jobs
Job	A group of steps that run on one runner
Step	A single command or action within a job
Artifact	A file output from a job, passed to later jobs
Cache	Persisted files to speed up future runs (node_modules, etc.)

Jobs within a pipeline can run **in parallel** or **sequentially** depending on dependencies.

Runners

GitHub-hosted runners:

- `ubuntu-latest` , `windows-latest` , `macos-latest`
- Free minutes on public repos; billed on private
- Fresh environment every run — no state leaks

GitLab shared runners:

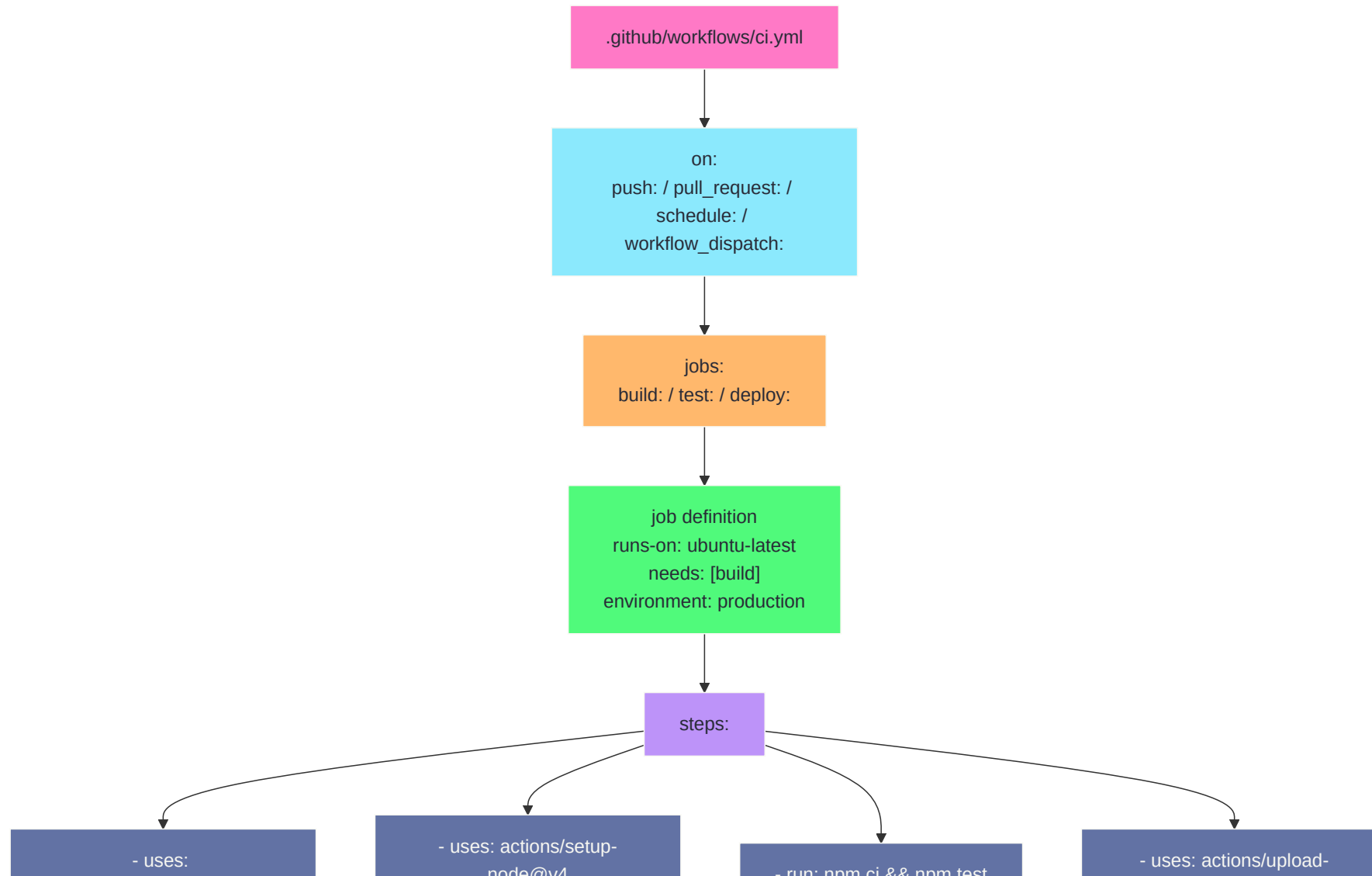
- Provided by GitLab SaaS on GitLab.com
- Docker executor by default

Self-hosted runners:

- Run on your own infrastructure
- Useful for: access to private networks, custom software, GPU workloads, cost savings

Part 4 — GitHub Actions

GitHub Actions — Workflow Structure



Minimal Workflow Example

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm ci
      - run: npm test
```


Triggers

```
on:
  push:
    branches: [main, 'release/*']
    paths: ['src/**', 'package.json']    # only run when these change

  pull_request:
    branches: [main]

  schedule:
    - cron: '0 6 * * 1'    # every Monday at 06:00 UTC

  workflow_dispatch:    # manual trigger with optional inputs
    inputs:
      environment:
        description: 'Target environment'
        required: true
        default: 'staging'
```

Secrets and Environment Variables

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      # Predefined context variables
      - run: echo "Commit SHA: ${github.sha}"
      - run: echo "Branch: ${github.ref_name}"

      # Repository secret
      - run: echo "${secrets.API_KEY}" | some-cli login

      # Set an env var for the rest of the job
      - run: echo "APP_VERSION=1.2.3" >> $GITHUB_ENV
      - run: echo "Deploying version $APP_VERSION"
```

Never `echo` a secret directly — it will be masked but it is still bad practice.

Caching Dependencies

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: ${ runner.os }-npm-${ hashFiles('**/package-lock.json') }
    restore-keys: |
      ${ runner.os }-npm-

- run: npm ci
```

How it works:

- First run: cache miss → `npm ci` runs → cache saved
- Subsequent runs: cache hit → restore → `npm ci` is fast (only changed packages)
- Cache key invalidates when `package-lock.json` changes

Matrix Builds

Test against multiple versions in parallel:

```
jobs:
  test:
    strategy:
      matrix:
        node-version: [18, 20, 22]
        os: [ubuntu-latest, windows-latest]
    runs-on: ${ matrix.os }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node-version }
      - run: npm ci && npm test
```

This creates **6 parallel jobs** (3 node versions × 2 OS).

Artifacts and Job Dependencies

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/

  deploy:
    needs: build # waits for build to succeed
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: dist
      - run: ./deploy.sh
```

Conditional Steps

```
steps:
  - run: npm test

  # Only run on main branch
  - name: Deploy to production
    if: github.ref == 'refs/heads/main'
    run: ./deploy-prod.sh

  # Only run on failure
  - name: Notify on failure
    if: failure()
    uses: slackapi/slack-github-action@v1
    with:
      payload: '{"text": "Build failed on ${github.ref}}"'
    env:
      SLACK_WEBHOOK_URL: ${secrets.SLACK_WEBHOOK }
```

Reusable Workflows

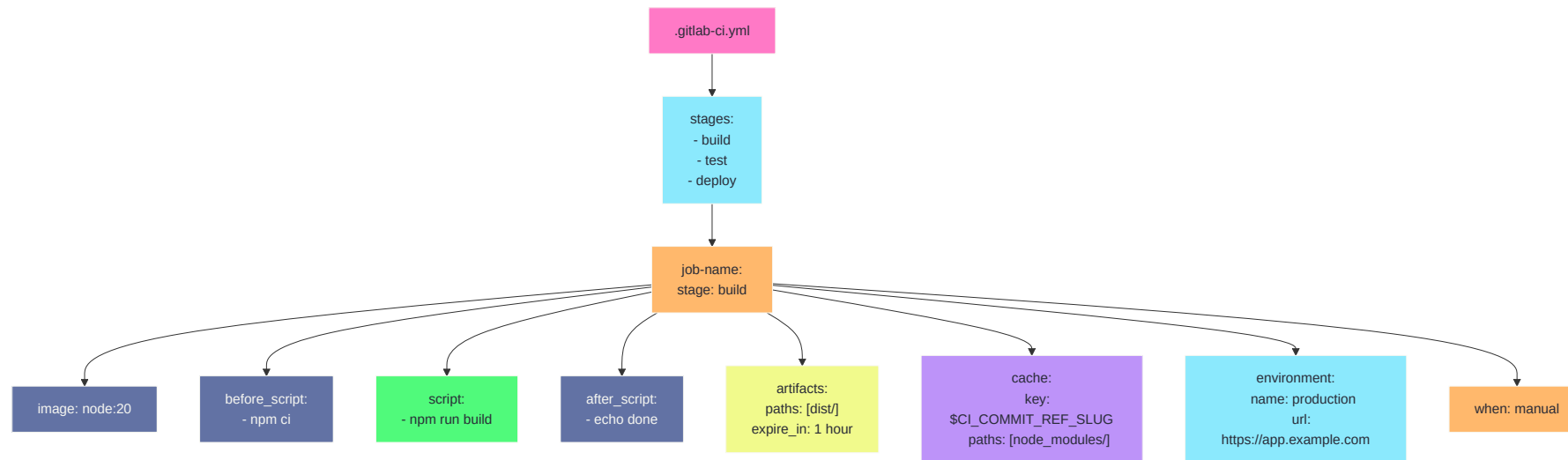
Shared workflow (`.github/workflows/shared-build.yml`):

```
on:
  workflow_call:
    inputs:
      node-version:
        type: string
        default: '20'

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ inputs.node-version }}
      - run: npm ci && npm test
```

Part 5 — GitLab CI

GitLab CI — Workflow Structure



Minimal .gitlab-ci.yml

```
stages:
  - build
  - test
  - deploy

build:
  stage: build
  image: node:20
  script:
    - npm ci
    - npm run build
  artifacts:
    paths:
      - dist/
    expire_in: 1 hour

test:
  stage: test
  image: node:20
  script:
    - npm ci
    - npm test
```

GitLab Variables

```
variables:
  APP_ENV: staging          # job-level variable

deploy:
  stage: deploy
  script:
    - echo "Deploying to $APP_ENV"
    - echo "Commit: $CI_COMMIT_SHA"
    - echo "Pipeline: $CI_PIPELINE_ID"
    - echo "Branch: $CI_COMMIT_REF_NAME"
    - echo "Secret: $DEPLOY_TOKEN" # masked variable set in UI
```

Predefined variables (always available):

Variable	Value
<code>\$CI_COMMIT_SHA</code>	Full commit hash
<code>\$CI_COMMIT_REF_NAME</code>	Branch or tag name

Cache and Artifacts

```
test:
  stage: test
  image: node:20
  cache:
    key: $CI_COMMIT_REF_SLUG          # per-branch cache
    paths:
      - node_modules/
  script:
    - npm ci
    - npm test
  artifacts:
    reports:
      junit: test-results.xml        # shown in GitLab MR UI
    paths:
      - coverage/
  expire_in: 30 days
```

Cache vs Artifacts:

include: and extends: — DRY Pipelines

```
# .gitlab-ci.yml
include:
  - project: 'org/shared-ci-templates'
    ref: main
    file: '/templates/node-build.yml'

# Use a template job
my-build:
  extends: .node-build-template
  variables:
    NODE_VERSION: '20'
```

GitLab CI Templates (built-in):

```
include:
  - template: Security/SAST.gitlab-ci.yml
  - template: Code-Quality.gitlab-ci.yml
```

Manual Gates and Environments

```
deploy-production:
  stage: deploy
  script:
    - ./deploy.sh production
  environment:
    name: production
    url: https://app.example.com
  when: manual # human must click "Run"
  only:
    - main
```

In the GitLab UI:

- The job appears in the pipeline with a  **play button**
- The environment tracks what was deployed and when
- You can roll back from the Environments page

GitHub Actions vs. GitLab CI

Feature	GitHub Actions	GitLab CI
Config file	<code>.github/workflows/*.yml</code>	<code>.gitlab-ci.yml</code>
Trigger keyword	<code>on:</code>	(implicit — on every push by default)
Execution unit	<code>job</code>	<code>job</code>
Parallel jobs	<code>strategy.matrix</code>	<code>parallel:</code> keyword
Sharing logic	<code>uses: org/repo/workflow.yml</code>	<code>include:</code> + <code>extends:</code>
Secrets	<code>\${{ secrets.NAME }}</code>	<code>\$VARIABLE_NAME</code>
Manual gate	<code>environment:</code> + protection rules	<code>when: manual</code>
Artifact	<code>upload_artifact</code> / <code>download</code>	

Day 1 — Summary

What we covered:

1. **DevOps culture** — silos vs. shared ownership, Three Ways, DORA metrics
2. **Git branching** — GitFlow, GitHub Flow, GitLab Flow, branch protection, SemVer, conventional commits
3. **CI/CD fundamentals** — CI vs. CD Delivery vs. CD Deployment, pipeline anatomy, runners
4. **GitHub Actions** — workflow structure, triggers, secrets, caching, matrix, artifacts, reusable workflows
5. **GitLab CI** — `.gitlab-ci.yml`, stages, variables, cache/artifacts, `include:` / `extends:`, manual gates

Tomorrow: